

Prueba Técnica — Full-Stack

Empresa: Crazy Imagine Software

Posición: Full-Stack

Tiempo estimado de trabajo: 8 a 12 horas

Plazo de entrega: 4 a 5 días desde la recepción de esta prueba

Nombre del candidato: _____

Fecha de envío: _____

Contexto

En Crazy Imagine construimos software a medida para clientes globales, y buena parte de ese software integra **automatizaciones** que conectan la aplicación con servicios externos. Para esta prueba vas a construir **CrazySupportHub**, una herramienta interna de tickets de soporte, cuyo corazón es una **integración bidireccional con n8n**: cuando se crea un ticket, tu aplicación dispara un flujo de n8n que lo **clasifica y enriquece**, y ese flujo le responde a tu aplicación con el resultado.

Queremos ver cómo diseñas una API, modelas datos, manejas autenticación y, sobre todo, cómo integras tu aplicación con un servicio orientado a eventos como n8n: tu app debe **responder** a lo que n8n le envía y **consumir** lo que el flujo produce.

Objetivo de la prueba

Evaluar tu nivel full-stack real de forma integral: diseño de API REST, autenticación y roles, modelado y persistencia de datos, integración con un servicio externo (n8n) en las dos direcciones, y un frontend funcional que maneja estado asíncrono con buena UX. Más que cantidad de features, buscamos **criterio**: decisiones técnicas sólidas, código limpio y una entrega que funcione de punta a punta y esté bien comunicada.

Vale más una entrega más pequeña pero pulida y funcional que una llena de features rotas.

Stack permitido

Fijamos el stack para que las entregas sean comparables, pero te dejamos elegir dentro de él.

Frontend:

- **React** (con Vite) o **Next.js**. TypeScript recomendado (suma, no obligatorio).

Backend:

- **Node.js** con **Express** o **NestJS** (elige el que domines). TypeScript recomendado.
- API **REST**.

Base de datos:

- **PostgreSQL**, mediante un ORM a tu elección (**Prisma**, **TypeORM** o **Sequelize**). Debe haber **persistencia real** (no datos en memoria).

Integración:

- **n8n** — puedes correrlo self-hosted (Docker o `npx n8n`) o usar n8n Cloud. El **workflow que construyas debes exportarlo como JSON** e incluirlo en el repo.

No permitido:

- Bases de datos que no sean relacionales/Postgres para el core (puedes usar Redis u otros como apoyo si lo justificas).
- Sustituir n8n por lógica embebida en tu backend: la clasificación/enriquecimiento **debe** ocurrir en un flujo de n8n. Ese es el punto de la prueba.

Si tienes dudas sobre si alguna herramienta encaja, escríbenos antes de invertir horas.

Arquitectura general

Tu entrega tiene **tres piezas** que se comunican:



1. El **frontend** crea el ticket llamando a tu **API**.
2. Tu **API** persiste el ticket (`enrichmentStatus: pending`) y **dispara un webhook** al flujo de n8n con los datos del ticket.
3. El **flujo de n8n** clasifica/enriquece el ticket y hace un **callback** a un endpoint de tu API.
4. Tu API guarda el enriquecimiento y el **frontend lo muestra** cuando el estado pasa a `done` .

Requisitos funcionales

Backend

1. **Autenticación**. Registro, login y **JWT**. Rutas protegidas. Passwords **hasheados** (bcrypt/argon2). Nunca guardes contraseñas en texto plano.
2. **Roles**. Dos roles: **admin** (ve y gestiona todos los tickets) y **agent** (ve los tickets que creó o tiene asignados). Aplica la autorización en las operaciones correspondientes.
3. **CRUD de tickets** con validación de entrada (campos requeridos, tipos, longitudes razonables).
4. **Listado avanzado** en el endpoint de tickets:
 - **Filtros** por `status` , `priority` y `category` .
 - **Búsqueda** por texto en título/descripción.

- **Paginación y ordenamiento.**
5. **Disparo saliente a n8n.** Al crear un ticket, tu backend hace `POST` al **webhook de n8n** (la URL va en una variable de entorno, no hardcodeada).
 6. **Endpoint de callback.** Expón `POST /api/webhooks/n8n/enrichment` (o ruta equivalente) que reciba el resultado del flujo. Debe estar **protegido con un secreto compartido** enviado en un header (p. ej. `X-Webhook-Secret`); rechaza la llamada si el secreto no coincide. Valida el payload y persiste el enriquecimiento en el ticket correspondiente.
 7. **Persistencia** en Postgres con **migraciones** y un **seed** (usuarios de prueba + tickets iniciales; puedes partir del `tickets-seed.json` que te damos).
 8. **Manejo de errores** consistente y **status codes HTTP correctos** (400, 401, 403, 404, 409, 500 según corresponda).
 9. **Tests automatizados** de al menos los endpoints clave: login, creación de ticket y callback de enriquecimiento.

Frontend

1. **Login y rutas protegidas.** Manejo del token, redirección si no hay sesión.
2. **Lista de tickets** con los **filtros, búsqueda y paginación** que expone tu API.
3. **Formulario de creación** de ticket con **validación inline** (errores debajo de cada campo, no `alert()`).
4. **Detalle de ticket** que muestra el enriquecimiento (prioridad, categoría, tags y, si aplica, la respuesta sugerida) y que **refleja el estado asíncrono**: cuando el ticket aún está `pending`, muéstralo, y actualiza la vista cuando pase a `done` (con `refetch` o `polling`; no hace falta WebSockets).
5. **Estados de UI cuidados**: loading, vacío (empty) y error.

n8n (el flujo)

1. Construye un **workflow** que:
 - Reciba el **webhook** con los datos del ticket.
 - **Clasifique** `priority` y `category` (ver más abajo los valores válidos).
 - Haga un **callback autenticado** a tu API con el resultado.
2. La **clasificación base puede ser por reglas** (p. ej. keywords → categoría/prioridad) dentro de n8n. Eso es suficiente para cumplir el requisito.
3. **Exporta el workflow como JSON** e inclúyelo en el repo (p. ej. `/n8n/workflow.json`), con instrucciones de setup en el README.

Modelo de datos (referencia)

No te imponemos el esquema exacto, pero estos son los campos esperados y sus valores válidos:

Ticket

- `title`, `description`
- `status`: `open` · `in_progress` · `resolved` · `closed`
- `createdBy`, `assignedTo` (usuario, `assignedTo` puede ser nulo)

- **Campos de enriquecimiento (los llena n8n):**

- `priority` : `low` · `medium` · `high` · `urgent` (nulo hasta enriquecer)
- `category` : `billing` · `technical` · `account` · `other` (nulo hasta enriquecer)
- `tags` : lista de strings
- `suggestedReply` : texto opcional (solo si usas el nodo de IA)
- `enrichmentStatus` : `pending` · `processing` · `done` · `failed`
- `enrichedAt` : fecha/hora

User

- `name` , `email` (único), `passwordHash` , `role` (`admin` · `agent`)

Datos de arranque

Junto a este documento recibes `tickets-seed.json` con usuarios de prueba (un admin y dos agentes) y ~12 tickets ficticios. Úsalo como punto de partida para tu seed. Los tickets ya traen ejemplos con `enrichmentStatus`: `pending` (aún sin clasificar) y `done` (ya enriquecidos), para que veas ambos estados.

Las contraseñas de los usuarios de prueba vienen en texto plano **solo en el seed**; tu backend debe **hashearlas** al insertarlas.

Nodo de IA (opcional — bonus)

La clasificación por reglas ya cumple. Como **bonus**, puedes añadir en n8n un **nodo de IA** que clasifique el ticket y genere una `suggestedReply` (respuesta sugerida al cliente). Consideraciones:

- Es **opcional**: no penaliza no usarlo, y no queremos que gastes en API keys si no quieres.
- **No comitees secretos**. Las API keys van en variables de entorno / credenciales de n8n, nunca en el repo.

Requisitos no funcionales

- **README** con:
 - **Setup de las tres piezas** (frontend, backend, n8n): cómo instalar, variables de entorno necesarias y cómo correr todo en local.
 - **Decisiones técnicas** (1-2 párrafos): qué elegiste y por qué (Express vs NestJS, ORM, cómo modelaste el enriquecimiento, cómo manejas el estado asíncrono en el front).
 - **Uso de IA** (ver reglas más abajo).
 - **Pendientes / bugs conocidos** (si aplica): qué quedó sin terminar y por qué.
- **docker-compose recomendado** (Postgres + app; n8n puede ir en compose o correrse aparte) para que corramos tu entrega fácilmente. No es obligatorio, pero suma.
- **Seguridad básica**: passwords hasheados, JWT, secreto para el callback de n8n, sin secretos en el repo, validación de entrada.

- **Historial de commits coherente** que refleje tu proceso. **Un único commit final del tipo "todo listo" baja la nota** — no refleja un proceso de trabajo real.

Entregables

1. **Repositorio en GitHub** (público, o privado con acceso para el evaluador) que incluya: **frontend**, **backend**, **workflow de n8n exportado (JSON)** y **README**. El repo debe poder clonarse y correrse siguiendo el README.
2. **Video/GIF corto (obligatorio)**. Una grabación breve (1-3 min) mostrando el **flujo end-to-end**: creas un ticket → el flujo de n8n corre → el enriquecimiento aparece en la UI. Es tu prueba de que la integración funciona de verdad. Sube el video al repo o comparte un enlace (Loom, Drive, YouTube no listado, etc.).
3. **Deploy funcional (opcional — bonus)**. Si puedes, despliega la app (front + back) en Railway, Render, Fly.io, Vercel o similar. n8n puede quedarse en local con instrucciones. No es obligatorio, pero es un buen plus.

Rúbrica de evaluación

Te mostramos los pesos exactos con los que vamos a calificar. Úsalos para priorizar dónde poner energía.

Bloque	Peso
Integración n8n (bidireccional + workflow)	25%
Backend: API, auth, arquitectura	25%
Frontend: funcionalidad + UX	20%
Modelo de datos y persistencia	10%
Calidad de código y tests	10%
Git y entrega (README, video, decisiones)	10%
Bonus opcional (nodo de IA, deploy, docker-compose, roles finos, extras con criterio)	+5%
Total	100% + 5% bonus

Tres cosas importantes sobre cómo aplicamos esta rúbrica:

- **No esperamos perfección.** Buscamos código limpio, criterio en las decisiones y una integración que funcione de punta a punta.
- **Si no terminas alguna feature, documéntalo en el README** con qué falta y por qué. Mejor 4 features sólidas y reconocer la quinta como pendiente, que 5 features rotas.
- **El bonus solo cuenta si el resto está bien.** No sacrifiques lo obligatorio por sumar bonus.

Reglas y consideraciones

Uso de IA — permitido con transparencia

A diferencia de otras pruebas nuestras, aquí **puedes usar IA como herramienta de trabajo** (Cursor, Claude, ChatGPT, Copilot, etc.). Saber trabajar con IA es parte del oficio. A cambio te pedimos:

- **Transparencia:** documenta en el README qué hiciste con ayuda de IA y qué a mano. Ser honesto nunca te perjudica.
- **Responsabilidad:** el resultado es tuyo. "La IA lo escribió así" no es excusa si algo se cae o no sabes explicarlo.
- **Defensa en entrevista:** en la entrevista técnica posterior te pediremos **explicar tu código, hacer cambios en vivo y justificar decisiones** (incluido el flujo de n8n). Si no puedes hacerlo con fluidez, se nota.

Preguntas

Si algo no queda claro en este documento, **escríbenos antes de asumir**. Mejor preguntar que invertir horas en una dirección equivocada. Las consultas a nuestro contacto no afectan tu calificación.

Cómo entregar

Cuando hayas terminado, envíanos un email a support@crazyimagine.com con:

- **Asunto:** Prueba Full-Stack – [Tu nombre completo]
- **Cuerpo del email:**
 - URL del repositorio en GitHub.
 - URL o ubicación del **video demo**.
 - URL del deploy (si lo hiciste).
 - Cualquier nota adicional que consideres relevante (opcional).

Plazo: 4 a 5 días desde la recepción de esta prueba. Si por algún motivo no vas a llegar al plazo, escríbenos al mismo email **antes** de que se cumpla; podemos coordinar una extensión razonable. No avisar y entregar tarde sí afecta la evaluación.